# Imlementarea algoritmilor genetici

Detecția fraudei bancare prin selecția regulilor și optimizarea pragurilor

**Rezumat**

În acest curs implementăm de la zero, folosind doar Python de bază (modulul `random`) și Matplotlib pentru vizualizare, un algoritm genetic care construiește automat un detector de fraudă pentru tranzacții bancare. Cromozomul codifică simultan *ce reguli sunt folosite* (selecția trăsăturilor) și *la ce praguri se declanșează*. Funcția de fitness reflectă un cost economic real: fraudele ratate costă, alertele false costă, iar regulile inutile sunt penalizate. Fiecare pas este explicat, codificat și însoțit de o reprezentare grafică.

## Cuprins

**1 Obiectivele cursului** **1**

**2 Problema: detecția fraudei cu reguli** **2**

**3 Algoritmul genetic pe scurt** **2**

**4 Modelarea problemei** **2**
4.1 Trăsăturile unei tranzacții . . . . . 2
4.2 Cromozomul . . . . . 3
4.3 Funcția de fitness . . . . . 3

**5 Implementare pas cu pas** **3**

**6 Ce am învățat și limitări** **12**

**A Codul complet** **13**

## 1 Obiectivele cursului

La finalul cursului vei putea:

- să explici componentele unui algoritm genetic (populație, cromozom, fitness, selecție, încrucișare, mutație, elitism);
- să modelezi o problemă reală ca problemă de optimizare cu cromozomi *micști* (biți + valori reale);
- să implementezi fiecare operator în Python de bază, fără biblioteci specializate;
- să vizualizezi convergența și să interpretezi soluția obținută;
- să evaluezi corect soluția pe date de test și să discuți limitările metodei.

**Cerințe.** Python 3.8+ și `matplotlib`. Nu folosim `numpy`, `pandas` sau `scikit-learn`, tocmai pentru ca fiecare linie de cod să fie transparentă.

## 2 Problema: detecția fraudei cu reguli

O bancă primește mii de tranzacții pe oră. Analiștii umani pot verifica doar o mică parte dintre ele, deci un sistem automat *semnalează* tranzacțiile suspecte. Un sistem clasic de tip *rule-based* funcționează astfel:

$$\text{semnalează tranzacția } t \iff \exists\, i \text{ activă astfel încât } r_i(t, \theta_i) = \text{adevărat},$$

unde fiecare regulă $r_i$ compară o trăsătură cu un prag $\theta_i$ (de exemplu: *suma* $\geq \theta_1$).

Inginerii stabilesc de obicei pragurile manual, ceea ce are două probleme:

1. pragurile sunt alese „din experiență", nu optimizate;
2. nu este clar *care* reguli merită păstrate: fiecare regulă în plus prinde mai multe fraude, dar generează și mai multe alerte false.

Aceasta este o problemă de **optimizare combinatorială și constituie în același timp** – un teren bun pentru un algoritm genetic.

**Ideea centrală:** Lăsăm evoluția să caute combinația de reguli și praguri care *minimizează costul total* al băncii: pierderile din fraude nedetectate plus costul verificării alertelor.

## 3 Algoritmul genetic pe scurt

Un algoritm genetic (AG) menține o *populație* de soluții candidate (*indivizi*) și o îmbunătățește iterativ, imitând evoluția naturală.

| **Concept** | **În problema noastră** |
|---|---|
| Individ / cromozom | Un detector complet (reguli active + praguri) |
| Genă | O regulă: perechea (bit de activare, prag) |
| Populație | 40 de detectoare candidate |
| Fitness | Fracțiunea din pierdere evitată (mai mare = mai bun) |
| Selecție | Turneu între 3 indivizi aleși la întâmplare |
| Încrucișare | Schimb de reguli întregi între doi părinți |
| Mutație | Comutarea unui bit sau perturbarea unui prag |
| Elitism | Cei mai buni 2 indivizi trec neschimbați mai departe |

Schema generală (pseudocod):

1. Generează aleator populația inițială.
2. **Repetă** pentru $G$ generații:
   a) evaluează fitness-ul fiecărui individ;
   b) copiază elita în generația următoare;
   c) până la completarea populației: alege doi părinți (selecție), încrucișează-i, mută descendenții;
   d) înlocuiește populația.
3. Returnează cel mai bun individ întâlnit.

## 4 Modelarea problemei

### 4.1 Trăsăturile unei tranzacții

Fiecare tranzacție are șase trăsături:

| Idx | Trăsătură | Regulă | Interval prag |
|---|---|---|---|
| 0 | suma (RON) | suma $\geq \theta$ | [50, 2000] |
| 1 | ora (0–23) | ora $\leq \theta$ (noapte) | [0, 8] |
| 2 | distanța față de domiciliu (km) | dist. $\geq \theta$ | [5, 500] |
| 3 | nr. tranzacții în ultima oră | nr. $\geq \theta$ | [1, 10] |
| 4 | riscul comerciantului (0–1) | risc $\geq \theta$ | [0.1, 0.95] |
| 5 | vechimea contului (zile) | vechime $\leq \theta$ | [1, 3000] |

Ultima trăsătură (vechimea contului) a fost generată *aproape neinformativ*: ne așteptăm ca algoritmul să o elimine singur. Acesta este momentul în care AG face **selecția trăsăturilor**.

### 4.2 Cromozomul

Un individ este o listă de 6 gene, iar fiecare genă este o pereche $(b_i, \theta_i)$:

$$\text{individ} = [(b_1, \theta_1), (b_2, \theta_2), \ldots, (b_6, \theta_6)], \qquad b_i \in \{0, 1\},\ \theta_i \in [\theta_i^{\min}, \theta_i^{\max}].$$

Bitul $b_i$ decide dacă regula $i$ este folosită, iar $\theta_i$ este pragul ei. Un astfel de cromozom se numește *mixt* (binar + real).

### 4.3 Funcția de fitness

Pentru un set de tranzacții notăm cu $TP$, $FP$, $FN$ numărul de fraude prinse, alerte false și fraude ratate. Costul total este

$$C = \underbrace{\sum_{t \in FN} \text{suma}_t}_{\text{pierderi din fraude ratate}} + \underbrace{c_{rev}\,(TP + FP)}_{\text{verificarea alertelor}} + \underbrace{c_{reg} \cdot \textstyle\sum_i b_i}_{\text{penalizare de complexitate}},$$

cu $c_{rev} = 5$ RON per alertă și $c_{reg} = 150$ RON per regulă activă. Fără niciun detector, banca pierde $C_0 = \sum_{t \text{ fraudă}} \text{suma}_t$. Definim fitness-ul ca fracțiunea din pierdere *evitată*:

$$\text{fitness} = 1 - \frac{C}{C_0}.$$

Un fitness de 0.9 înseamnă că detectorul evită 90% din pierderea potențială (după costurile de operare). Dacă $C > C_0$, fitness-ul devine negativ: detectorul ar fi mai prost decât să nu facem nimic.

**De ce penalizăm regulile active?** Fără termenul $c_{reg}$, algoritmul nu are niciun motiv să renunțe la o regulă inutilă. Penalizarea implementează următorul principiu: între două soluții la fel de bune, preferăm pe cea mai simplă.

## 5 Implementare pas cu pas

Fiecare secvență de cod de mai jos se pune, în ordine, într-un singur fișier Python. Scriptul complet se găsește și în Anexa A.

### Pasul 0 – Importuri și reproductibilitate

Folosim doar `random` și `matplotlib`. Fixăm sămânța generatorului aleator pentru ca toți cursanții să obțină aceleași rezultate.

```
import random
import matplotlib.pyplot as plt

random.seed(42)  # rezultate reproductibile
```

## Pasul 1 – Generarea datelor sintetice

Datele reale despre fraudă sunt confidențiale, deci simulăm tranzacții. Fraudele ($\approx 6\%$) au, în medie, sume mai mari, apar mai des noaptea, de la distanțe mari, la comercianți riscanți și în rafale. Folosim `lognormvariate`, `expovariate` și `betavariate` din modulul `random`.

Împărțim datele în **antrenare** (70%, pe care evoluează AG) și **test** (30%, folosit doar la final). Această separare este esențială: fără ea nu am ști dacă detectorul generalizează.

```
FRAUD_RATE = 0.06        # ~6% dintre tranzactii sunt frauduloase
N_TRANZACTII = 2400

def genereaza_tranzactie(este_frauda):
    """Returneaza [suma, ora, distanta_km, nr_tranz_ora, risc_comerciant, vechime_cont]."""
    if este_frauda:
        suma = random.lognormvariate(5.3, 0.9)
        if random.random() < 0.65:
            ora = random.randint(0, 5)          # noaptea
        else:
            ora = random.randint(6, 23)
        distanta = random.expovariate(1 / 150)  # departe de domiciliu
        nr_tranz = 1 + int(random.expovariate(1 / 2.5))
        risc = random.betavariate(5, 3)         # comercianti riscanti
        vechime = random.randint(1, 2000)       # aproape neinformativ
    else:
        suma = random.lognormvariate(3.6, 1.0)
        ora = max(0, min(23, int(random.gauss(14, 4))))
        distanta = random.expovariate(1 / 15)
        nr_tranz = int(random.expovariate(1 / 0.8))
        risc = random.betavariate(2, 8)
        vechime = random.randint(30, 3000)
    return [suma, ora, distanta, nr_tranz, risc, vechime]

def genereaza_set_date(n):
    date = []
    for _ in range(n):
        este_frauda = random.random() < FRAUD_RATE
        date.append((genereaza_tranzactie(este_frauda), 1 if este_frauda else 0))
    return date

toate = genereaza_set_date(N_TRANZACTII)
taiere = int(0.7 * len(toate))
date_antrenare = toate[:taiere]
date_test = toate[taiere:]

n_fraude = sum(y for _, y in date_antrenare)
print("Antrenare:", len(date_antrenare), "tranzactii,", n_fraude, "fraude")
print("Test:     ", len(date_test), "tranzactii,",
      sum(y for _, y in date_test), "fraude")
```

```
Antrenare: 1680 tranzactii, 102 fraude
Test:      720 tranzactii, 30 fraude
```

**Date dezechilibrate** Doar aproximativ 6% dintre tranzacții sunt fraude. Din acest motiv nu folosim *acuratețea* ca scop: un detector care nu semnalează nimic ar avea 94% acurateță și ar fi complet inutil. Costul economic este mult mai potrivit.

## Pasul 2 – Vizualizarea datelor

Înainte de orice optimizare, ne uităm la date. Histogramele arată de ce regulile pe bază de prag pot funcționa: fraudele au sume și distanțe mult mai mari și se concentrează noaptea.

```
def coloana(date, idx, eticheta):
```

```
    return [x[idx] for x, y in date if y == eticheta]

fig, axe = plt.subplots(1, 3, figsize=(13, 3.6))
setari = [(0, "Suma tranzactiei (RON)", 1500),
          (2, "Distanta fata de domiciliu (km)", 600),
          (1, "Ora tranzactiei", 24)]

for ax, (idx, titlu, limita) in zip(axe, setari):
    legit = [v for v in coloana(date_antrenare, idx, 0) if v <= limita]
    fraud = [v for v in coloana(date_antrenare, idx, 1) if v <= limita]
    ax.hist(legit, bins=30, density=True, alpha=0.6, label="legitime")
    ax.hist(fraud, bins=30, density=True, alpha=0.6, label="frauduloase")
    ax.set_title(titlu)
    ax.set_ylabel("densitate")
axe[0].legend()
plt.tight_layout()
plt.savefig("fig1_date.png", dpi=150)
plt.show()
```

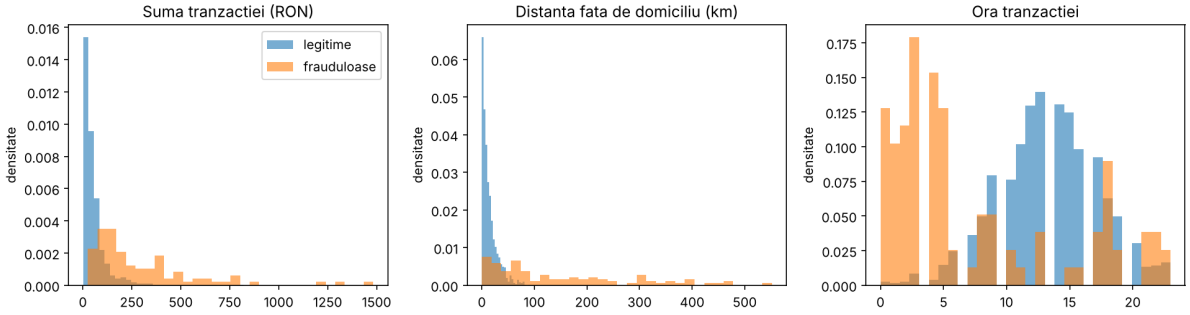



Figura 1: Distribuțiile trăsăturilor pentru tranzacții legitime și frauduloase (setul de antrenare).

## Pasul 3 – Reguli și reprezentarea cromozomului

Definim cele șase reguli într-o listă de dicționare: trăsătura folosită, sensul comparației și intervalul în care poate varia pragul. Apoi scriem funcția `semnaleaza`, care decide dacă un detector (un individ) marchează o tranzacție ca suspectă. Logica este un **SAU**: este suficientă o singură regulă activă declanșată.

```
# Fiecare regula: "daca trasatura (idx) >= / <= prag, semnaleaza tranzactia"
REGULI = [
    {"nume": "suma mare",          "idx": 0, "sens": ">=", "min": 50,  "max": 2000},
    {"nume": "ora de noapte",      "idx": 1, "sens": "<=", "min": 0,   "max": 8},
    {"nume": "distanta mare",      "idx": 2, "sens": ">=", "min": 5,   "max": 500},
    {"nume": "tranzactii rapide",  "idx": 3, "sens": ">=", "min": 1,   "max": 10},
    {"nume": "comerciant riscant", "idx": 4, "sens": ">=", "min": 0.1, "max": 0.95},
    {"nume": "cont nou",           "idx": 5, "sens": "<=", "min": 1,   "max": 3000},
]

# Un individ = lista de 6 gene; fiecare gena = [bit_activ, prag]
exemplu = [[1, 800.0], [0, 3.0], [1, 120.0], [0, 4.0], [0, 0.7], [0, 365.0]]

def semnaleaza(individ, x):
    """True daca CEL PUTIN O regula activa se declanseaza (SAU logic)."""
    for i in range(len(REGULI)):
        bit, prag = individ[i]
        if bit == 1:
            regula = REGULI[i]
```

```
            valoare = x[regula["idx"]]
            if regula["sens"] == ">=" and valoare >= prag:
                return True
            if regula["sens"] == "<=" and valoare <= prag:
                return True
    return False
```

### Pasul 4 – Funcția de fitness

Parcurgem tranzacțiile, numărăm $TP$, $FP$, $FN$ și calculăm costul definit în secțiunea de modelare. Funcția `evalueaza` returnează toate statisticile (le vom refolosi la test), iar `fitness` extrage doar valoarea scalară.

```
COST_REVIZUIRE = 5.0      # RON per tranzactie semnalata (analist uman)
COST_REGULA = 150.0       # penalizare per regula activa (complexitate)

def evalueaza(individ, date):
    tp = fp = fn = 0
    pierdut = 0.0         # suma fraudelor ratate
    baza = 0.0            # pierderea totala daca nu am avea niciun detector
    for x, y in date:
        semnalata = semnaleaza(individ, x)
        if y == 1:
            baza += x[0]
        if semnalata and y == 1:
            tp += 1
        elif semnalata and y == 0:
            fp += 1
        elif (not semnalata) and y == 1:
            fn += 1
            pierdut += x[0]
    nr_reguli = sum(bit for bit, _ in individ)
    cost = pierdut + COST_REVIZUIRE * (tp + fp) + COST_REGULA * nr_reguli
    return {"tp": tp, "fp": fp, "fn": fn, "nr_reguli": nr_reguli,
            "cost": cost, "fitness": 1.0 - cost / baza}

def fitness(individ, date):
    return evalueaza(individ, date)["fitness"]

print("Fitness exemplu:", round(fitness(exemplu, date_antrenare), 3))
```

```
Fitness exemplu: 0.596
```

Chiar un individ ales „de mână” obține un fitness de aproximativ 0.6; AG va trebui să îl îmbunătățească.

### Pasul 5 – Populația inițială

Generăm indivizi complet aleatori: fiecare bit este 0 sau 1, iar fiecare prag este tras uniform din intervalul permis. Funcția `copiaza` este necesară deoarece listele din Python sunt mutabile: fără o copie *profundă* a genelor, mutația unui copil ar modifica și părintele.

```
def individ_aleator():
    return [[random.randint(0, 1), random.uniform(r["min"], r["max"])]
            for r in REGULI]

def populatie_initiala(dimensiune):
    return [individ_aleator() for _ in range(dimensiune)]

def copiaza(individ):
    return [[bit, prag] for bit, prag in individ]
```

### Pasul 6 – Selecția prin turneu

Alegem la întâmplare $k = 3$ indivizi și îl păstrăm pe cel cu fitness maxim. Metoda are avantaje practice: nu cere ca fitness-ul să fie pozitiv (spre deosebire de selecția „ruletă") și presiunea de selecție se reglează prin $k$. Un $k$ mare converge rapid, dar pierde diversitate.

```
def selectie_turneu(populatie, fitnessuri, k=3):
    candidati = random.sample(range(len(populatie)), k)
    castigator = max(candidati, key=lambda i: fitnessuri[i])
    return populatie[castigator]
```

### Pasul 7 – Încrucișarea

Aplicăm o *încrucișare uniformă pe gene*: pentru fiecare regulă, cu probabilitate 0.5 cei doi copii își schimbă între ei perechea $(b_i, \theta_i)$. Schimbăm bitul și pragul *împreună*, ca o regulă să rămână coerentă: un prag potrivit pentru „suma mare" nu are sens atașat unei reguli de „oră".

```
def incrucisare(p1, p2, prob=0.8):
    c1, c2 = copiaza(p1), copiaza(p2)
    if random.random() < prob:
        for i in range(len(c1)):
            if random.random() < 0.5:        # schimbam intreaga regula
                c1[i], c2[i] = c2[i], c1[i]
    return c1, c2
```

### Pasul 8 – Mutația

Mutația reintroduce diversitate. Avem două tipuri, corespunzătoare celor două componente ale genei:

- **bit-flip** (probabilitate 0.10): activează sau dezactivează o regulă;
- **perturbare gaussiană** (probabilitate 0.20): deplasează pragul cu un zgomot $\mathcal{N}(0, \sigma^2)$, unde $\sigma$ este 10% din lățimea intervalului; apoi tăiem valoarea în interval.

```
def mutatie(individ, p_bit=0.10, p_prag=0.20, sigma=0.10):
    for i, regula in enumerate(REGULI):
        if random.random() < p_bit:
            individ[i][0] = 1 - individ[i][0]          # activeaza / dezactiveaza
        if random.random() < p_prag:
            amplitudine = sigma * (regula["max"] - regula["min"])
            nou = individ[i][1] + random.gauss(0, amplitudine)
            individ[i][1] = max(regula["min"], min(regula["max"], nou))
    return individ
```

### Pasul 9 – Bucla evolutivă

Aici asamblăm toate piesele. Pentru fiecare generație: evaluăm populația, reținem cel mai bun individ (global), păstrăm istoricul pentru grafice, copiem elita și completăm populația cu descendenți.

Calculăm și, la fiecare generație, *fracțiunea populației în care fiecare regulă este activă*. Aceasta ne va arăta mai târziu cum acționează selecția trăsăturilor.

```
DIM_POP = 40
N_GEN = 40
N_ELITE = 2
```

```python

def algoritm_genetic(date):
    populatie = populatie_initiala(DIM_POP)
    istoric = {"best": [], "medie": [], "activ": []}
    best_global, best_fit = None, -1e9

    for gen in range(N_GEN):
        fit = [fitness(ind, date) for ind in populatie]
        ordine = sorted(range(DIM_POP), key=lambda i: fit[i], reverse=True)

        if fit[ordine[0]] > best_fit:
            best_fit = fit[ordine[0]]
            best_global = copiaza(populatie[ordine[0]])

        istoric["best"].append(fit[ordine[0]])
        istoric["medie"].append(sum(fit) / DIM_POP)
        istoric["activ"].append(
            [sum(ind[i][0] for ind in populatie) / DIM_POP
             for i in range(len(REGULI))])

        # elitism: cei mai buni N_ELITE trec direct in generatia urmatoare
        noua = [copiaza(populatie[i]) for i in ordine[:N_ELITE]]
        while len(noua) < DIM_POP:
            p1 = selectie_turneu(populatie, fit)
            p2 = selectie_turneu(populatie, fit)
            c1, c2 = incrucisare(p1, p2)
            noua.append(mutatie(c1))
            if len(noua) < DIM_POP:
                noua.append(mutatie(c2))
        populatie = noua

    return best_global, best_fit, istoric
```

## Pasul 10 – Rularea algoritmului

```python
best, best_fit, istoric = algoritm_genetic(date_antrenare)

def descrie(individ):
    for i, (bit, prag) in enumerate(individ):
        regula = REGULI[i]
        stare = "ACTIVA " if bit == 1 else "inactiva"
        print(f"  {regula['nume']:20s} {stare}  {regula['sens']} {prag:8.2f}")

print("Cel mai bun fitness (antrenare):", round(best_fit, 4))
descrie(best)
```

```
Cel mai bun fitness (antrenare): 0.9473
  suma mare            inactiva  >=  1092.33
  ora de noapte        ACTIVA    <=     5.14
  distanta mare        ACTIVA    >=   109.56
  tranzactii rapide    inactiva  >=     8.21
  comerciant riscant   ACTIVA    >=     0.47
  cont nou             inactiva  <=  1655.85
```

Soluția găsită este interpretabilă: *semnalează tranzacțiile din timpul nopții (ora* $\leq 5$*), de la peste* $\approx 110$ *km față de domiciliu sau la comercianți cu risc* $\geq 0.47$. Regula inutilă („cont nou”) a fost eliminată, iar regulile „suma mare” și „tranzacții rapide” au fost dezactivate pentru că nu aduceau beneficii suplimentare față de costul lor.

## Pasul 11 – Convergența

```
generatii = list(range(1, N_GEN + 1))
plt.figure(figsize=(7, 4))
plt.plot(generatii, istoric["best"], label="cel mai bun individ", linewidth=2)
plt.plot(generatii, istoric["medie"], label="media populatiei", linestyle="--")
plt.xlabel("Generatia")
plt.ylabel("Fitness (fractiune din pierdere evitata)")
plt.title("Evolutia fitness-ului")
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig("fig2_evolutie.png", dpi=150)
plt.show()
```

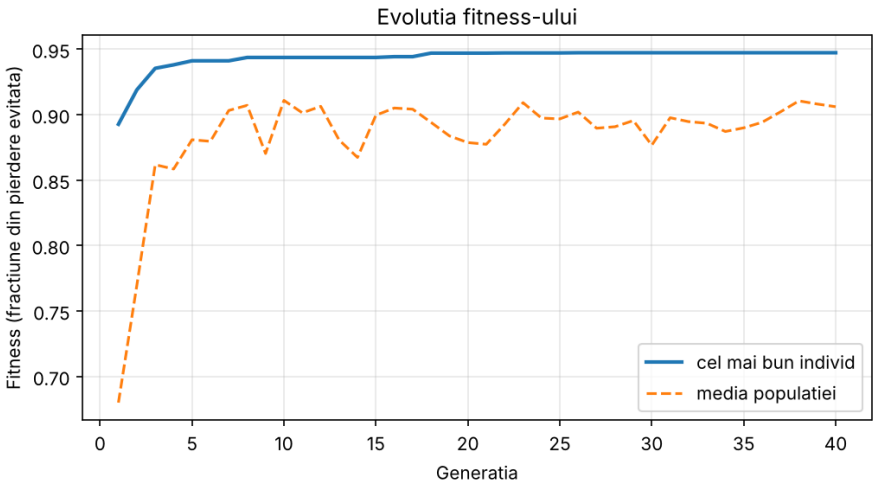



Figura 2: Fitness-ul celui mai bun individ (linie continuă) și media populației (linie punctată).

Observații importante:

- cel mai bun fitness este *monoton nedescrescător*: nu scade niciodată, datorită elitismului;
- media populației urcă rapid în primele $\approx 5$ generații, apoi oscilează: mutația și încrucișarea continuă să producă indivizi mai slabi, ceea ce este normal și chiar util pentru explorare;
- după $\approx 18$ de generații îmbunătățirile devin marginale.

## Pasul 12 – Evaluarea pe setul de test

Acum folosim datele pe care AG nu le-a văzut niciodată. Comparăm soluția evoluată cu două detectoare de referință:

- **Expert**: o regulă manuală plauzibilă (suma $\geq 1000$ RON sau distanța $\geq 300$ km);
- **Toate regulile**: toate cele 6 reguli active, cu praguri „rezonabile”.

```
# Doua detectoare de referinta, pentru comparatie
expert = [[1, 1000.0], [0, 4.0], [1, 300.0], [0, 5.0], [0, 0.7], [0, 365.0]]
toate_regulile = [[1, 300.0], [1, 5.0], [1, 100.0], [1, 4.0], [1, 0.6], [1, 365.0]]

detectoare = {"Expert\n(suma/distanta)": expert,
              "Toate\nregulile": toate_regulile,
              "Algoritm\ngenetic": best}

nume, savings, precizii, recalluri = [], [], [], []
for n, ind in detectoare.items():
```

```
    r = evalueaza(ind, date_test)
    precizie = r["tp"] / (r["tp"] + r["fp"]) if (r["tp"] + r["fp"]) > 0 else 0.0
    recall = r["tp"] / (r["tp"] + r["fn"])
    nume.append(n)
    savings.append(r["fitness"])
    precizii.append(precizie)
    recalluri.append(recall)
    print(n.replace("\n", " "), "-> fitness:", round(r["fitness"], 3),
          "| precizie:", round(precizie, 3), "| recall:", round(recall, 3),
          "| alerte:", r["tp"] + r["fp"])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
ax1.bar(nume, savings, color=["#999999", "#bbbbbb", "#2a7fba"])
ax1.set_title("Pierdere evitata pe setul de test")
ax1.set_ylabel("Fitness")

pozitii = range(len(nume))
latime = 0.38
ax2.bar([p - latime / 2 for p in pozitii], precizii, latime, label="precizie")
ax2.bar([p + latime / 2 for p in pozitii], recalluri, latime, label="recall")
ax2.set_xticks(list(pozitii))
ax2.set_xticklabels(nume)
ax2.set_ylim(0, 1.05)
ax2.set_title("Precizie si recall (test)")
ax2.legend()
plt.tight_layout()
plt.savefig("fig3_comparatie.png", dpi=150)
plt.show()
```

```
Expert (suma/distanta) -> fitness: 0.282 | precizie: 0.875 | recall: 0.233 | alerte: 8
Toate regulile -> fitness: 0.797 | precizie: 0.196 | recall: 0.967 | alerte: 148
Algoritm genetic -> fitness: 0.898 | precizie: 0.42 | recall: 0.967 | alerte: 69
```

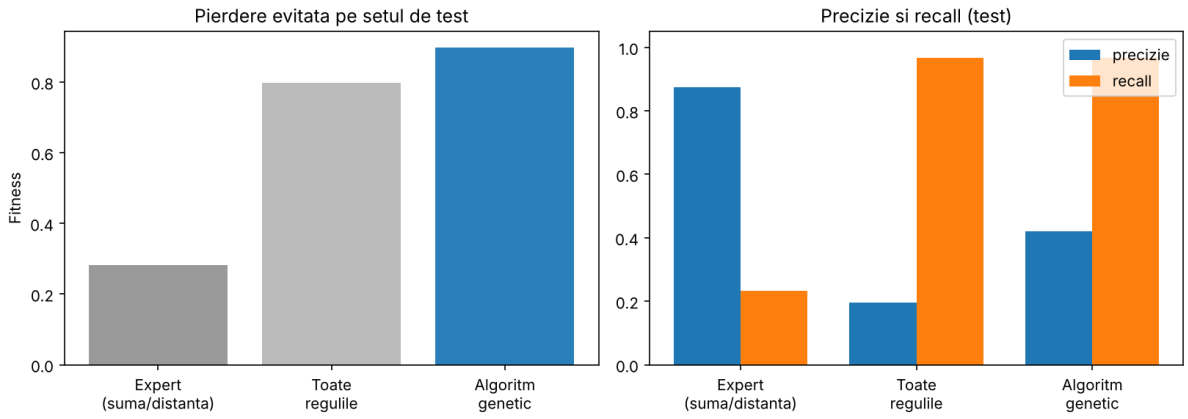



Figura 3: Comparație pe setul de test: pierdere evitată (stânga), precizie și recall (dreapta).

Interpretare:

- detectorul **expert** are precizie mare (87.5%), dar prinde doar 23% din fraude: este prea conservator, iar banca pierde bani;
- **toate regulile** prind aproape toate fraudele (96.7%), dar generează 148 de alerte, dintre care doar $\approx 20\%$ sunt reale: analiștii sunt copleșiți;
- **AG** păstrează același recall (96.7%) cu aproape jumătate din alerte (69) și obține cel mai bun fitness pe test (0.898).

Faptul că fitness-ul pe test (0.898) este mai mic decât pe antrenare (0.947) este normal:

soluția s-a adaptat, în parte, la particularitățile setului de antrenare. Diferența mică indică o generalizare bună. Cu doar 30 de fraude în setul de test, aceste cifre au însă o incertitudine considerabilă.

### Pasul 13 – Ce reguli supraviețuiesc?

Acesta este graficul care ilustrează *selecția trăsăturilor*: urmărim, la fiecare generație, ce fracțiune din populație folosește fiecare regulă.

```
numele_regulilor = [r["nume"] for r in REGULI]
plt.figure(figsize=(7.5, 4))
for i, nume_regula in enumerate(numele_regulilor):
    serie = [istoric["activ"][g][i] for g in range(N_GEN)]
    plt.plot(generatii, serie, label=nume_regula, linewidth=2)
plt.xlabel("Generatia")
plt.ylabel("Fractiunea populatiei cu regula activa")
plt.title("Ce reguli supravietuiesc selectiei?")
plt.ylim(-0.02, 1.02)
plt.grid(alpha=0.3)
plt.legend(fontsize=8, loc="center left", bbox_to_anchor=(1, 0.5))
plt.tight_layout()
plt.savefig("fig4_reguli.png", dpi=150)
plt.show()
```

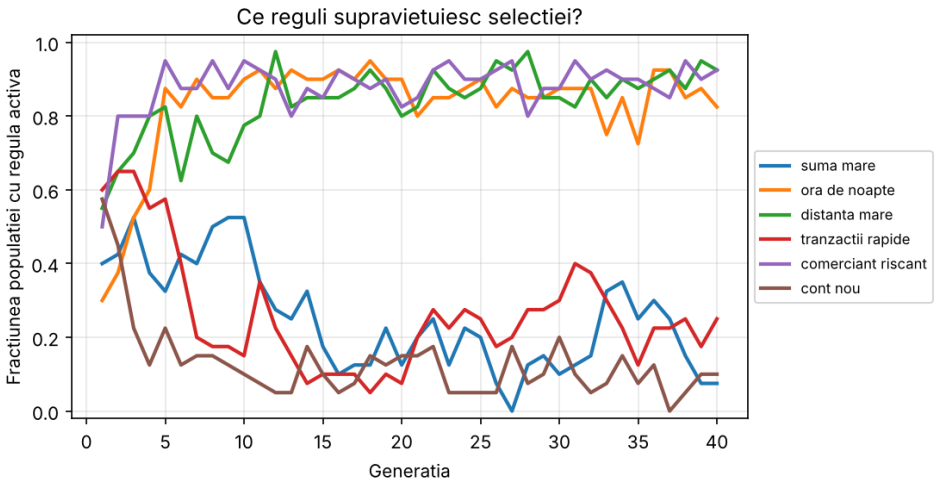



Figura 4: Fracțiunea populației în care fiecare regulă este activă, pe generații.

În primele generații toate regulile apar cu frecvențe între 30% și 60% (efect al inițializării aleatoare). Pe parcurs, regulile *ora de noapte*, *distanța mare* și *comerciantul riscant* ajung la 80–95%, în timp ce *contul nou* (zgomot) dispare aproape complet. AG a descoperit singur care trăsături sunt informative.

### Pasul 14 – Pragurile în spațiul trăsăturilor

Ca ultimă vizualizare, punem tranzacțiile de test într-un grafic *sumă* × *distanță* (scară logaritmică) și desenăm pragurile detectorului evoluat, atunci când regulile respective sunt active.

```
fig, ax = plt.subplots(figsize=(7, 5))
legit = [(x[0], x[2]) for x, y in date_test if y == 0]
fraud = [(x[0], x[2]) for x, y in date_test if y == 1]
ax.scatter([p[0] for p in legit], [p[1] for p in legit],
           s=8, alpha=0.35, label="legitime")
```

```python
ax.scatter([p[0] for p in fraud], [p[1] for p in fraud],
           s=22, color="crimson", label="frauduloase")

bit_suma, prag_suma = best[0]
bit_dist, prag_dist = best[2]
if bit_suma == 1:
    ax.axvline(prag_suma, color="k", linestyle="--", label="prag suma")
if bit_dist == 1:
    ax.axhline(prag_dist, color="g", linestyle="--", label="prag distanta")

ax.set_xscale("log")
ax.set_yscale("log")
ax.set_xlabel("Suma (RON, scara log)")
ax.set_ylabel("Distanta (km, scara log)")
ax.set_title("Pragurile invatate, pe setul de test")
ax.legend()
plt.tight_layout()
plt.savefig("fig5_scatter.png", dpi=150)
plt.show()
```

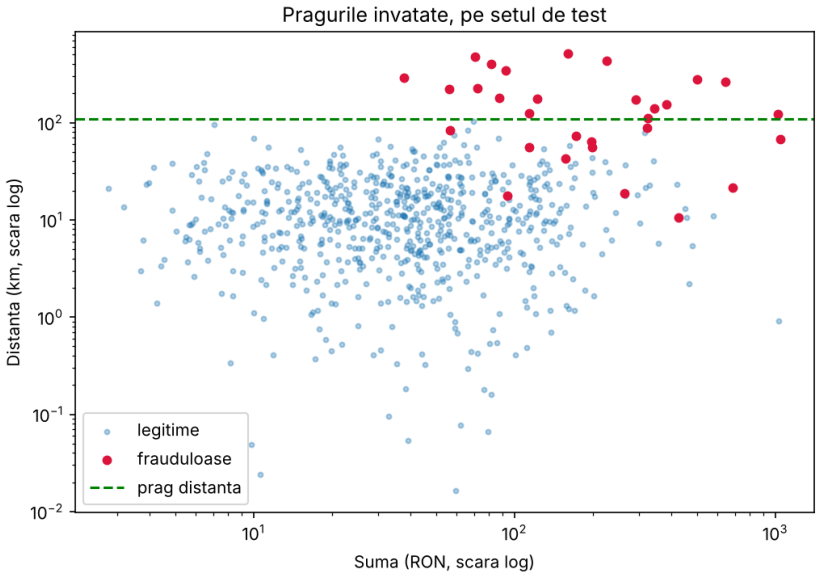



Figura 5: Tranzacțiile din setul de test în spațiul sumă–distanță. Liniile indică pragurile regulilor active (dacă este cazul).

Dacă regula pentru sumă este dezactivată (ca în rularea noastră), apare doar linia pragului de distanță. Fraudele cu distanță mică sunt prinse de celelalte reguli (noapte, comerciant riscant), nu de aceasta.

## 6 Ce am învățat și limitări

**Ce a făcut bine algoritmul genetic.**

- a optimizat simultan o componentă *discretă* (ce reguli) și una *continuă* (ce praguri), fără gradient și fără a cere funcției de cost să fie derivabilă;
- a încorporat direct un obiectiv economic (costuri asimetrice);
- a produs un model *interpretabil*, adică un set scurt de reguli ce pot fi auditate de un analist sau de un regulator.

**Limitări de care trebuie să ții cont.**

- **Date sintetice.** Rezultatele ilustrează metoda, nu performanța pe fraude reale, care sunt mai variate și se schimbă în timp (*concept drift*).
- **Variabilitate.** AG este stocastic: două rulări cu semințe diferite dau soluții diferite. În practică se rulează de mai multe ori și se compară rezultatele.
- **Test mic.** Cu 30 de fraude, diferențe mici între detectoare nu sunt semnificative statistic; ar fi necesară validare încrucișată.
- **Cost.** Evaluarea fitness-ului parcurge tot setul de date; pentru milioane de tranzacții ar trebui vectorizată (de exemplu cu `numpy`).
- **Parametri.** Dimensiunea populației, ratele de mutație sau costurile $c_{rev}$ și $c_{reg}$ influențează rezultatul și trebuie alese cu grijă.

# A Codul complet

Scriptul de mai jos reunește toți pașii, în ordinea în care trebuie executați. Figurile sunt salvate în directorul curent ca fișiere PNG (pe acestea le include documentul LaTeX).

```python
import random
import matplotlib.pyplot as plt

random.seed(42)  # rezultate reproductibile

FRAUD_RATE = 0.06        # ~6% dintre tranzactii sunt frauduloase
N_TRANZACTII = 2400

def genereaza_tranzactie(este_frauda):
    """Returneaza [suma, ora, distanta_km, nr_tranz_ora, risc_comerciant, vechime_cont]."""
    if este_frauda:
        suma = random.lognormvariate(5.3, 0.9)
        if random.random() < 0.65:
            ora = random.randint(0, 5)          # noaptea
        else:
            ora = random.randint(6, 23)
        distanta = random.expovariate(1 / 150)  # departe de domiciliu
        nr_tranz = 1 + int(random.expovariate(1 / 2.5))
        risc = random.betavariate(5, 3)         # comercianti riscanti
        vechime = random.randint(1, 2000)       # aproape neinformativ
    else:
        suma = random.lognormvariate(3.6, 1.0)
        ora = max(0, min(23, int(random.gauss(14, 4))))
        distanta = random.expovariate(1 / 15)
        nr_tranz = int(random.expovariate(1 / 0.8))
        risc = random.betavariate(2, 8)
        vechime = random.randint(30, 3000)
    return [suma, ora, distanta, nr_tranz, risc, vechime]

def genereaza_set_date(n):
    date = []
    for _ in range(n):
        este_frauda = random.random() < FRAUD_RATE
        date.append((genereaza_tranzactie(este_frauda), 1 if este_frauda else 0))
    return date

toate = genereaza_set_date(N_TRANZACTII)
taiere = int(0.7 * len(toate))
date_antrenare = toate[:taiere]
date_test = toate[taiere:]

n_fraude = sum(y for _, y in date_antrenare)
print("Antrenare:", len(date_antrenare), "tranzactii,", n_fraude, "fraude")
```

```python
print("Test:     ", len(date_test), "tranzactii,",
      sum(y for _, y in date_test), "fraude")

def coloana(date, idx, eticheta):
    return [x[idx] for x, y in date if y == eticheta]

fig, axe = plt.subplots(1, 3, figsize=(13, 3.6))
setari = [(0, "Suma tranzactiei (RON)", 1500),
          (2, "Distanta fata de domiciliu (km)", 600),
          (1, "Ora tranzactiei", 24)]

for ax, (idx, titlu, limita) in zip(axe, setari):
    legit = [v for v in coloana(date_antrenare, idx, 0) if v <= limita]
    fraud = [v for v in coloana(date_antrenare, idx, 1) if v <= limita]
    ax.hist(legit, bins=30, density=True, alpha=0.6, label="legitime")
    ax.hist(fraud, bins=30, density=True, alpha=0.6, label="frauduloase")
    ax.set_title(titlu)
    ax.set_ylabel("densitate")
axe[0].legend()
plt.tight_layout()
plt.savefig("fig1_date.png", dpi=150)
plt.show()

# Fiecare regula: "daca trasatura (idx) >= / <= prag, semnaleaza tranzactia"
REGULI = [
    {"nume": "suma mare",          "idx": 0, "sens": ">=", "min": 50,  "max": 2000},
    {"nume": "ora de noapte",      "idx": 1, "sens": "<=", "min": 0,   "max": 8},
    {"nume": "distanta mare",      "idx": 2, "sens": ">=", "min": 5,   "max": 500},
    {"nume": "tranzactii rapide",  "idx": 3, "sens": ">=", "min": 1,   "max": 10},
    {"nume": "comerciant riscant", "idx": 4, "sens": ">=", "min": 0.1, "max": 0.95},
    {"nume": "cont nou",           "idx": 5, "sens": "<=", "min": 1,   "max": 3000},
]

# Un individ = lista de 6 gene; fiecare gena = [bit_activ, prag]
exemplu = [[1, 800.0], [0, 3.0], [1, 120.0], [0, 4.0], [0, 0.7], [0, 365.0]]

def semnaleaza(individ, x):
    """True daca CEL PUTIN O regula activa se declanseaza (SAU logic)."""
    for i in range(len(REGULI)):
        bit, prag = individ[i]
        if bit == 1:
            regula = REGULI[i]
            valoare = x[regula["idx"]]
            if regula["sens"] == ">=" and valoare >= prag:
                return True
            if regula["sens"] == "<=" and valoare <= prag:
                return True
    return False

COST_REVIZUIRE = 5.0      # RON per tranzactie semnalata (analist uman)
COST_REGULA = 150.0       # penalizare per regula activa (complexitate)

def evalueaza(individ, date):
    tp = fp = fn = 0
    pierdut = 0.0         # suma fraudelor ratate
    baza = 0.0            # pierderea totala daca nu am avea niciun detector
    for x, y in date:
        semnalata = semnaleaza(individ, x)
        if y == 1:
            baza += x[0]
        if semnalata and y == 1:
            tp += 1
        elif semnalata and y == 0:
```

```
            fp += 1
        elif (not semnalata) and y == 1:
            fn += 1
            pierdut += x[0]
    nr_reguli = sum(bit for bit, _ in individ)
    cost = pierdut + COST_REVIZUIRE * (tp + fp) + COST_REGULA * nr_reguli
    return {"tp": tp, "fp": fp, "fn": fn, "nr_reguli": nr_reguli,
            "cost": cost, "fitness": 1.0 - cost / baza}

def fitness(individ, date):
    return evalueaza(individ, date)["fitness"]

print("Fitness exemplu:", round(fitness(exemplu, date_antrenare), 3))

def individ_aleator():
    return [[random.randint(0, 1), random.uniform(r["min"], r["max"])]
            for r in REGULI]

def populatie_initiala(dimensiune):
    return [individ_aleator() for _ in range(dimensiune)]

def copiaza(individ):
    return [[bit, prag] for bit, prag in individ]

def selectie_turneu(populatie, fitnessuri, k=3):
    candidati = random.sample(range(len(populatie)), k)
    castigator = max(candidati, key=lambda i: fitnessuri[i])
    return populatie[castigator]

def incrucisare(p1, p2, prob=0.8):
    c1, c2 = copiaza(p1), copiaza(p2)
    if random.random() < prob:
        for i in range(len(c1)):
            if random.random() < 0.5:        # schimbam intreaga regula
                c1[i], c2[i] = c2[i], c1[i]
    return c1, c2

def mutatie(individ, p_bit=0.10, p_prag=0.20, sigma=0.10):
    for i, regula in enumerate(REGULI):
        if random.random() < p_bit:
            individ[i][0] = 1 - individ[i][0]          # activeaza / dezactiveaza
        if random.random() < p_prag:
            amplitudine = sigma * (regula["max"] - regula["min"])
            nou = individ[i][1] + random.gauss(0, amplitudine)
            individ[i][1] = max(regula["min"], min(regula["max"], nou))
    return individ

DIM_POP = 40
N_GEN = 40
N_ELITE = 2

def algoritm_genetic(date):
    populatie = populatie_initiala(DIM_POP)
    istoric = {"best": [], "medie": [], "activ": []}
    best_global, best_fit = None, -1e9

    for gen in range(N_GEN):
        fit = [fitness(ind, date) for ind in populatie]
        ordine = sorted(range(DIM_POP), key=lambda i: fit[i], reverse=True)

        if fit[ordine[0]] > best_fit:
            best_fit = fit[ordine[0]]
            best_global = copiaza(populatie[ordine[0]])
```

```
170
171         istoric["best"].append(fit[ordine[0]])
172         istoric["medie"].append(sum(fit) / DIM_POP)
173         istoric["activ"].append(
174             [sum(ind[i][0] for ind in populatie) / DIM_POP
175              for i in range(len(REGULI))])
176
177         # elitism: cei mai buni N_ELITE trec direct in generatia urmatoare
178         noua = [copiaza(populatie[i]) for i in ordine[:N_ELITE]]
179         while len(noua) < DIM_POP:
180             p1 = selectie_turneu(populatie, fit)
181             p2 = selectie_turneu(populatie, fit)
182             c1, c2 = incrucisare(p1, p2)
183             noua.append(mutatie(c1))
184             if len(noua) < DIM_POP:
185                 noua.append(mutatie(c2))
186         populatie = noua
187
188     return best_global, best_fit, istoric
189
190 best, best_fit, istoric = algoritm_genetic(date_antrenare)
191
192 def descrie(individ):
193     for i, (bit, prag) in enumerate(individ):
194         regula = REGULI[i]
195         stare = "ACTIVA " if bit == 1 else "inactiva"
196         print(f"  {regula['nume']:20s} {stare}  {regula['sens']} {prag:8.2f}")
197
198 print("Cel mai bun fitness (antrenare):", round(best_fit, 4))
199 descrie(best)
200
201 generatii = list(range(1, N_GEN + 1))
202 plt.figure(figsize=(7, 4))
203 plt.plot(generatii, istoric["best"], label="cel mai bun individ", linewidth=2)
204 plt.plot(generatii, istoric["medie"], label="media populatiei", linestyle="--")
205 plt.xlabel("Generatia")
206 plt.ylabel("Fitness (fractiune din pierdere evitata)")
207 plt.title("Evolutia fitness-ului")
208 plt.grid(alpha=0.3)
209 plt.legend()
210 plt.tight_layout()
211 plt.savefig("fig2_evolutie.png", dpi=150)
212 plt.show()
213
214 # Doua detectoare de referinta, pentru comparatie
215 expert = [[1, 1000.0], [0, 4.0], [1, 300.0], [0, 5.0], [0, 0.7], [0, 365.0]]
216 toate_regulile = [[1, 300.0], [1, 5.0], [1, 100.0], [1, 4.0], [1, 0.6], [1, 365.0]]
217
218 detectoare = {"Expert\n(suma/distanta)": expert,
219               "Toate\nregulile": toate_regulile,
220               "Algoritm\ngenetic": best}
221
222 nume, savings, precizii, recalluri = [], [], [], []
223 for n, ind in detectoare.items():
224     r = evalueaza(ind, date_test)
225     precizie = r["tp"] / (r["tp"] + r["fp"]) if (r["tp"] + r["fp"]) > 0 else 0.0
226     recall = r["tp"] / (r["tp"] + r["fn"])
227     nume.append(n)
228     savings.append(r["fitness"])
229     precizii.append(precizie)
230     recalluri.append(recall)
231     print(n.replace("\n", " "), "-> fitness:", round(r["fitness"], 3),
232           "| precizie:", round(precizie, 3), "| recall:", round(recall, 3),
```

```
          "| alerte:", r["tp"] + r["fp"])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
ax1.bar(nume, savings, color=["#999999", "#bbbbbb", "#2a7fba"])
ax1.set_title("Pierdere evitata pe setul de test")
ax1.set_ylabel("Fitness")

pozitii = range(len(nume))
latime = 0.38
ax2.bar([p - latime / 2 for p in pozitii], precizii, latime, label="precizie")
ax2.bar([p + latime / 2 for p in pozitii], recalluri, latime, label="recall")
ax2.set_xticks(list(pozitii))
ax2.set_xticklabels(nume)
ax2.set_ylim(0, 1.05)
ax2.set_title("Precizie si recall (test)")
ax2.legend()
plt.tight_layout()
plt.savefig("fig3_comparatie.png", dpi=150)
plt.show()

numele_regulilor = [r["nume"] for r in REGULI]
plt.figure(figsize=(7.5, 4))
for i, nume_regula in enumerate(numele_regulilor):
    serie = [istoric["activ"][g][i] for g in range(N_GEN)]
    plt.plot(generatii, serie, label=nume_regula, linewidth=2)
plt.xlabel("Generatia")
plt.ylabel("Fractiunea populatiei cu regula activa")
plt.title("Ce reguli supravietuiesc selectiei?")
plt.ylim(-0.02, 1.02)
plt.grid(alpha=0.3)
plt.legend(fontsize=8, loc="center left", bbox_to_anchor=(1, 0.5))
plt.tight_layout()
plt.savefig("fig4_reguli.png", dpi=150)
plt.show()

fig, ax = plt.subplots(figsize=(7, 5))
legit = [(x[0], x[2]) for x, y in date_test if y == 0]
fraud = [(x[0], x[2]) for x, y in date_test if y == 1]
ax.scatter([p[0] for p in legit], [p[1] for p in legit],
           s=8, alpha=0.35, label="legitime")
ax.scatter([p[0] for p in fraud], [p[1] for p in fraud],
           s=22, color="crimson", label="frauduloase")

bit_suma, prag_suma = best[0]
bit_dist, prag_dist = best[2]
if bit_suma == 1:
    ax.axvline(prag_suma, color="k", linestyle="--", label="prag suma")
if bit_dist == 1:
    ax.axhline(prag_dist, color="g", linestyle="--", label="prag distanta")

ax.set_xscale("log")
ax.set_yscale("log")
ax.set_xlabel("Suma (RON, scara log)")
ax.set_ylabel("Distanta (km, scara log)")
ax.set_title("Pragurile invatate, pe setul de test")
ax.legend()
plt.tight_layout()
plt.savefig("fig5_scatter.png", dpi=150)
plt.show()
```